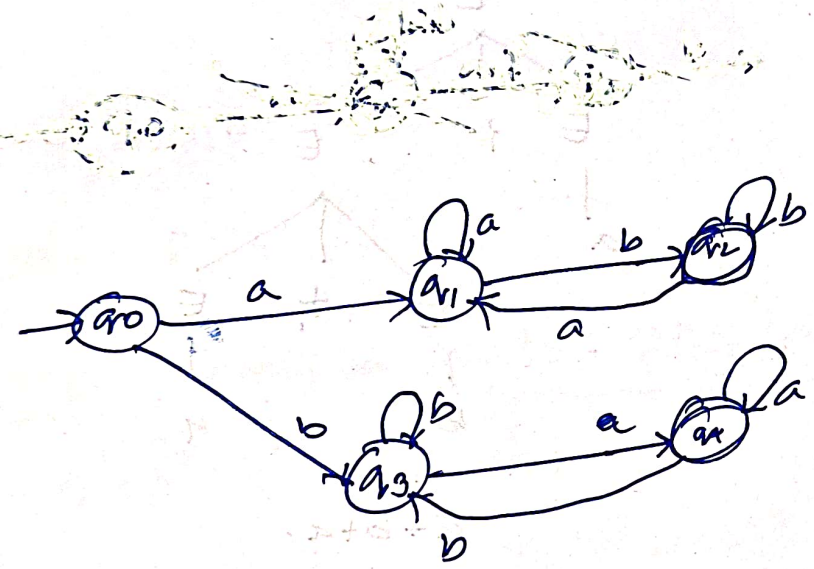
PART-A

1. NFA, Σ = {strings starting with 10 or 11} $\Sigma = \{0,1\}$

2. DFA for string which 1st and last letters do not match

$\Sigma = \{a,b\}$



3. Give a regular Expression for the set of all string not containing 101 as substring.

$0^*(1+00^*)^*0^*$

(4) state the closure properties for Regular Expression

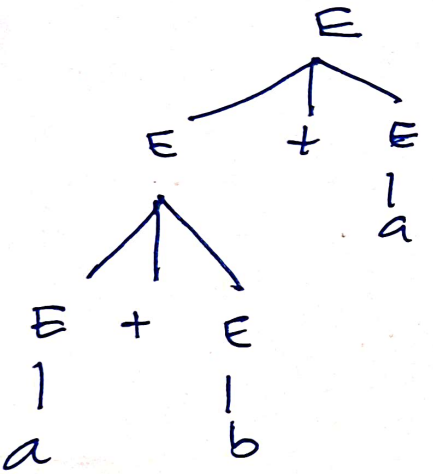
1. Homomorphism
2. Inverse homomorphism
3. Concatenation
4. Union
5. Intersection
6. Kleen closure
7. complement.

Q Explain with the help of an example of ambigious Grammar

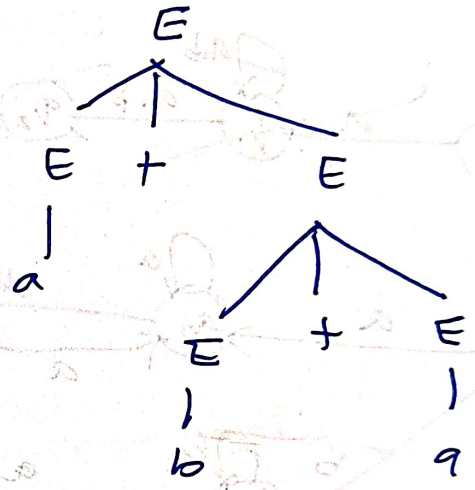
A Grammar G is said to be ambigious if there exist more than 1 left most derivations or right most derivation or parse tree. for the given input string.

eg: $E \rightarrow E+E \mid a \mid b$

final a+b+a



a+b+a



a+b+a.

Here both parse tree can derive a+b+a.

So it is a ambigious Grammars

6. Write CFG equivalent to the regular expression $0^*1(0+1)^*+1$

Ans: $G = (V, T, S, P)$

$V = S, A$

$T = 0, 1$

Start Symbol = S

Production Rules

1. $S \rightarrow A|1$ (covers 2 parts: $0^*1(0+1)^*$ and 1)
2. $A \rightarrow 0A|1A|1B|\epsilon$ (generates strings from $0^*1(0+1)^*$
3. $B \rightarrow 0B|1B|\epsilon$ (this handles $(0+1)^*$)

∴ the complete CFG for the regular expression $0^*1(0+1)^*+1$ is

$S \rightarrow A|1$

$A \rightarrow 0A|1A|1B|\epsilon$

$B \rightarrow 0B|1B|\epsilon$

7. What are the conditions required for push down automata to qualify as deterministic push down automata?

Ans: A PDA is a DPDA if there is never a choice for a next move in any instantaneous description

A PDA $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is deterministic if $\delta(q, a, x)$ has at most one member for any q in Q, a in $\Sigma \cup \{\epsilon\}$ and x in Γ

8. Can we construct a DPDA for the language ww^r. Justify your answer.

Ans: No we cannot construct a deterministic push down automata for language ww^r.

This is because ww^r is a context-free language that cannot be accepted

by a DPDA. A DPDA is a type of pushdown automata that accepts deterministic context free languages. A DPDA has atmost one legal transition for an input symbol.

The language ww^r requires determinism

9. Differentiate Recursive and Recursively Enumerable Languages

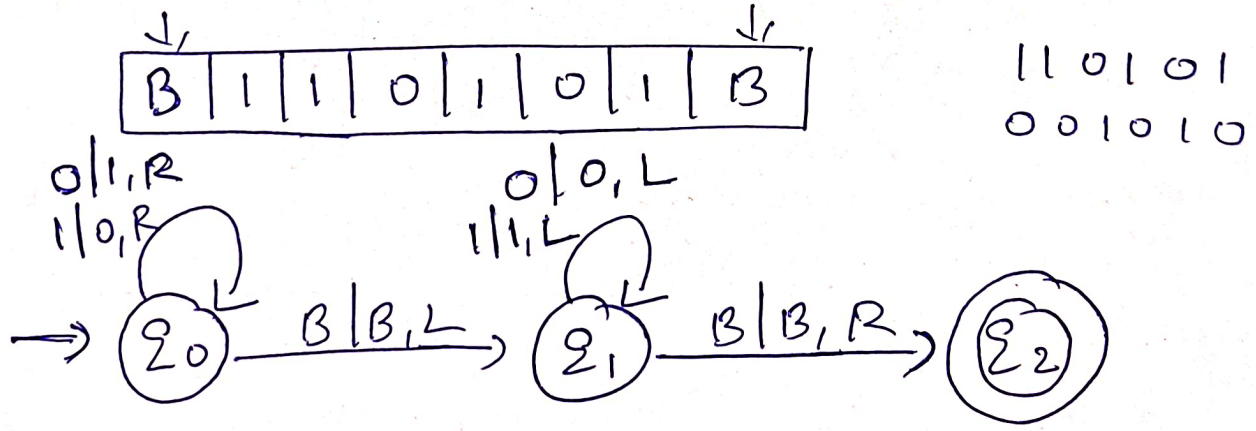
ns: Recursive languages

A language L is set to be recursive if there exists a Turing Machine which will accept all strings in L and reject all strings not in L. The TM will hold every time and give an answer accepted or rejected for each and every input string

Recursively enumerable languages-

A language L is said to be recursively enumerable language if there exists a Turing Machine which will accept and hold all input strings which are in L ~~but~~ but may or may not called for all input strings which are not in L.

10. Design a TM to find the 1's complement of a binary number.

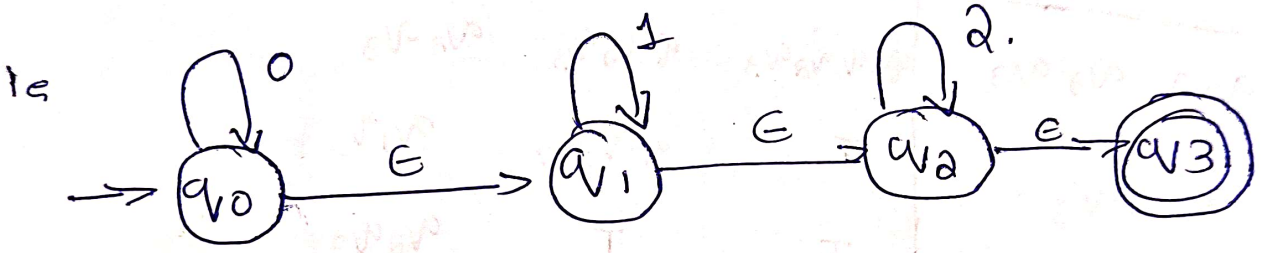


1 1 0 1 0 1
0 0 1 0 1 0

11 a) Construct an ϵ-NFA for the language $L = \{0^n 1^m 2^p \mid n, m, p \geq 0\}$ and convert it into equivalent DFA.

An ϵ-NFA is a non-deterministic finite automata which consist of ϵ states.

Given that $L = \{0^n 1^m 2^p \mid n, m, p \geq 0\}$



Transition table

q/Σ	0	1	2	ϵ
q_0	q_0	ϕ	ϕ	q_1
q_1	ϕ	q_1	ϕ	q_2
q_2	ϕ	ϕ	q_2	q_3
q_3	ϕ	ϕ	ϕ	ϕ

ϵ- closure of each state;

$\epsilon^*(q_0) = \{q_0, q_1, q_2, q_3\}$

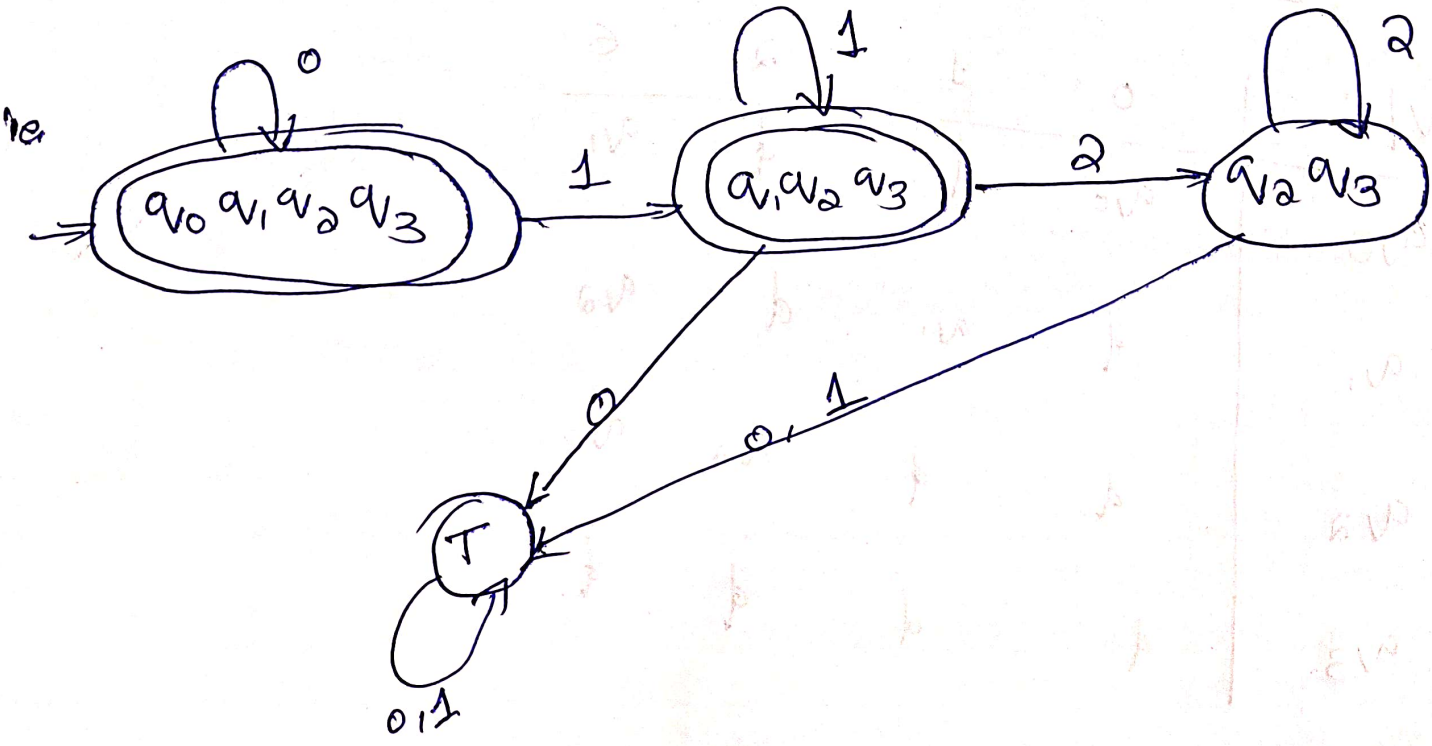
$\epsilon^*(q_1) = \{q_1, q_2, q_3\}$

$\epsilon^*(q_2) = \{q_2, q_3\}$

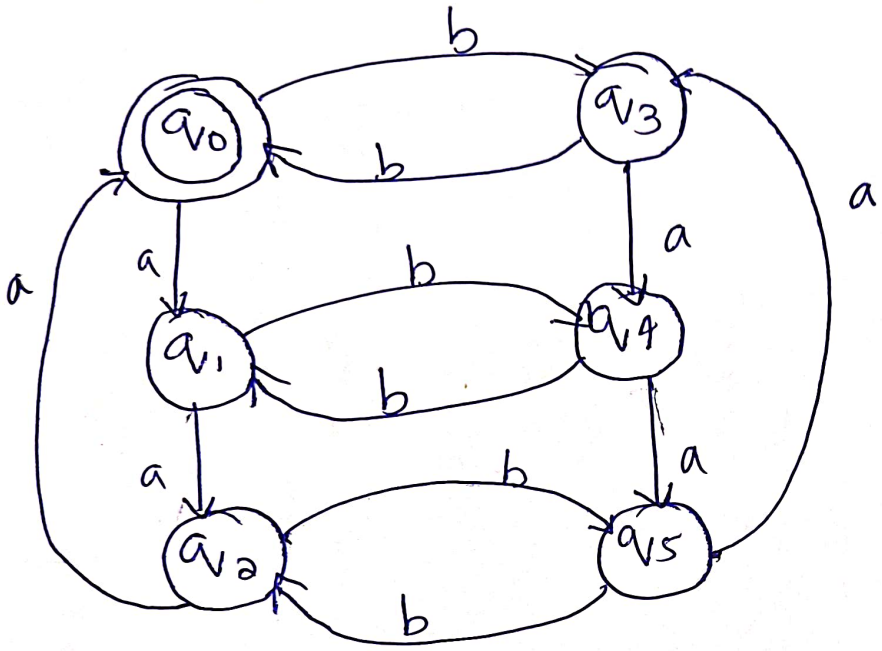
$\epsilon^*(q_3) = q_3$.

Converting to DFA.

	0	1	2
$q_0 q_1 q_2 q_3$	$q_0 q_1 q_2 q_3$	$q_1 q_2 q_3$	$q_2 q_3$
$q_1 q_2 q_3$	T	$q_1 q_2 q_3$	$q_2 q_3$
$q_2 q_3$	T	T	$q_2 q_3$



11b) Design a DFA for strings in which number of a's multiple of 3 and number of b's is multiple of 2. $\Sigma = \{a, b\}$



Formal defnition of DFA;

$M = (Q, \Sigma, \delta, q_0, f)$

Q : $\{q_0, q_1, q_2, q_3, q_4, q_5\}$

Σ : $\{a, b\}$

q_0 : q_0

f : q_0

Transition Table

Q/Σ	a	b
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_0	q_5
q_3	q_4	q_0
q_4	q_5	q_1
q_5	q_3	q_2

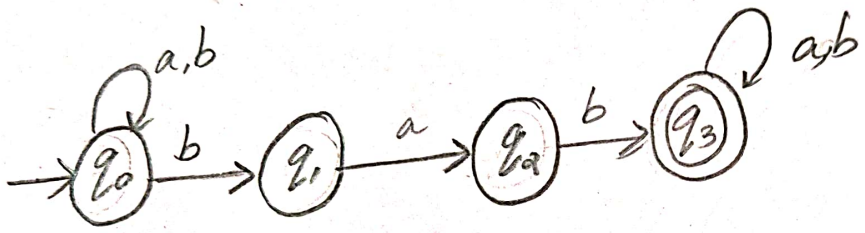
Draw the state-~~Diagram~~ transition diagram showing an NFA N for the following language L.

Obtain the DFA D equivalent to N by applying the subset construction method.

$$L = \{ x \in \{a,b\}^* / x \text{ contains 'bab' as a substring} \}.$$

State-transition Diagram

NFA,



Subset Construction method

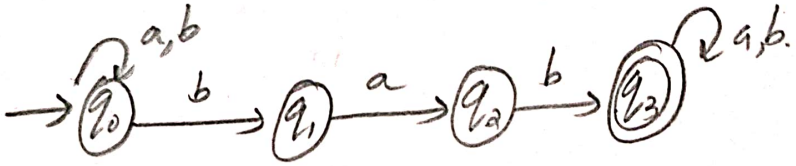
↓

NFA to DFA Conversion

NFA

$M = (Q, \Sigma, \delta, q_0, F)$

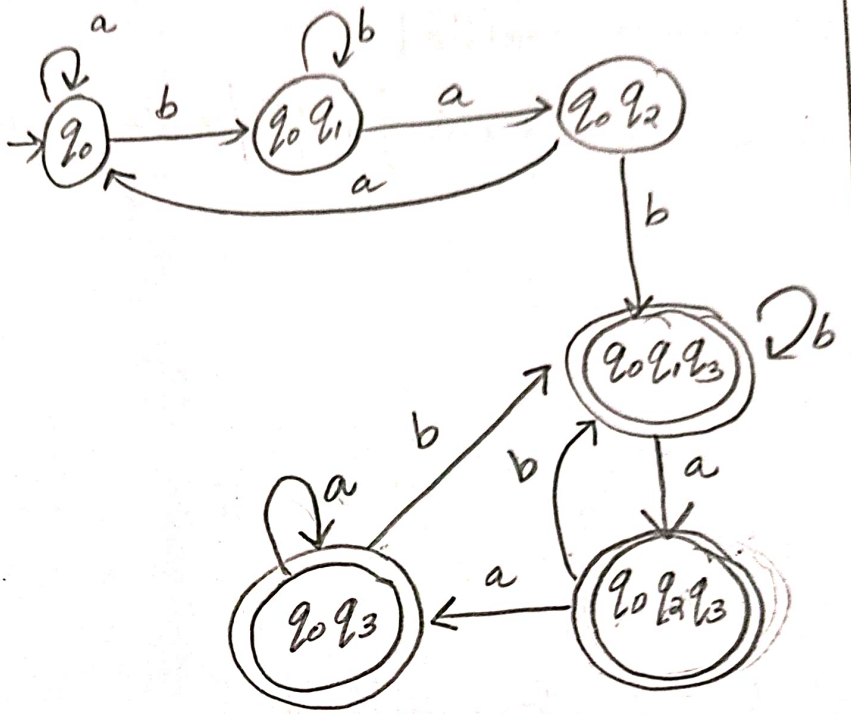
$\delta = Q \times \Sigma = 2^Q$



Q/Σ	a	b
q_0	q_0	$\{q_0, q_1\}$
q_1	q_2	ϕ
q_2	ϕ	q_3
q_3	q_3	q_3

Conversion To DFA

q/Σ	a	b
→ q_0	q_0	q_0q_1
q_0q_1	q_0q_2	q_0q_1
q_0q_2	q_0	$q_0q_1q_3$
$q_0q_1q_3$	$q_0q_2q_3$	$q_0q_1q_3$
$q_0q_2q_3$	q_0q_3	$q_0q_1q_3$
q_0q_3	q_0q_3	$q_0q_1q_3$



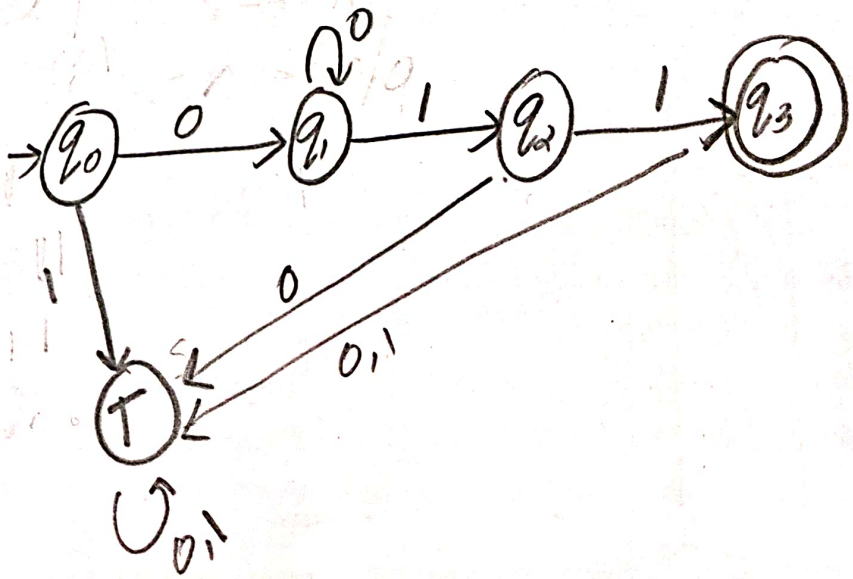
12.b. Construct a regular grammar for $L = \{0^n11 \mid n \geq 1\}$. Construct deterministic finite automata for the same.

$$G = (V, T, P, S)$$

$$\omega = \{011, 0011, 00011 \ldots\}$$

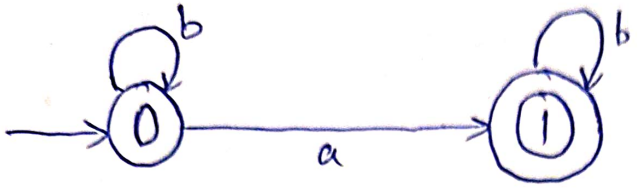
eg:1 $S \rightarrow A11$

$A \rightarrow 0A/0$



13.

a) Find the Regular Expression for the following DFA



Ans:

$$0 = \epsilon + 0b \quad \text{— (1)}$$

$$1 = 0a + 1b \quad \text{— (2)}$$

Using Arden's Theorem

$$R = Q + RP$$

$$R = QP^*$$

$$0 = \epsilon b^* = \underline{b^*} \quad \text{— (3)}$$

Sub (3) in (2)

$$1 = 0b^* b^* a + 1b$$

Using Arden's Theorem

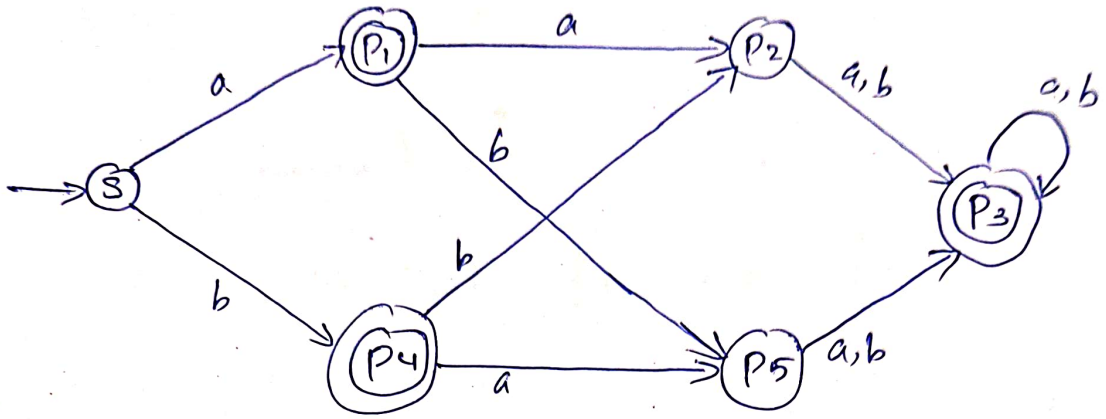
$$R = Q + RP$$

$$R = QP^*$$

$$1 = (b^* a) b^*$$

13.

b, Obtain the minimum state DFA from following DFA



Ans:

Q / Σ	a	b
→S	P_1	P_4
(P1)	P_2	P_5
P2	P_3	P_3
(P3)	P_3	P_3
(P4)	P_5	P_2
P5	P_3	P_3

0 equivalence

$\{S, P_2, P_5\}$ $\{P_1, P_3, P_4\}$

1 equivalence

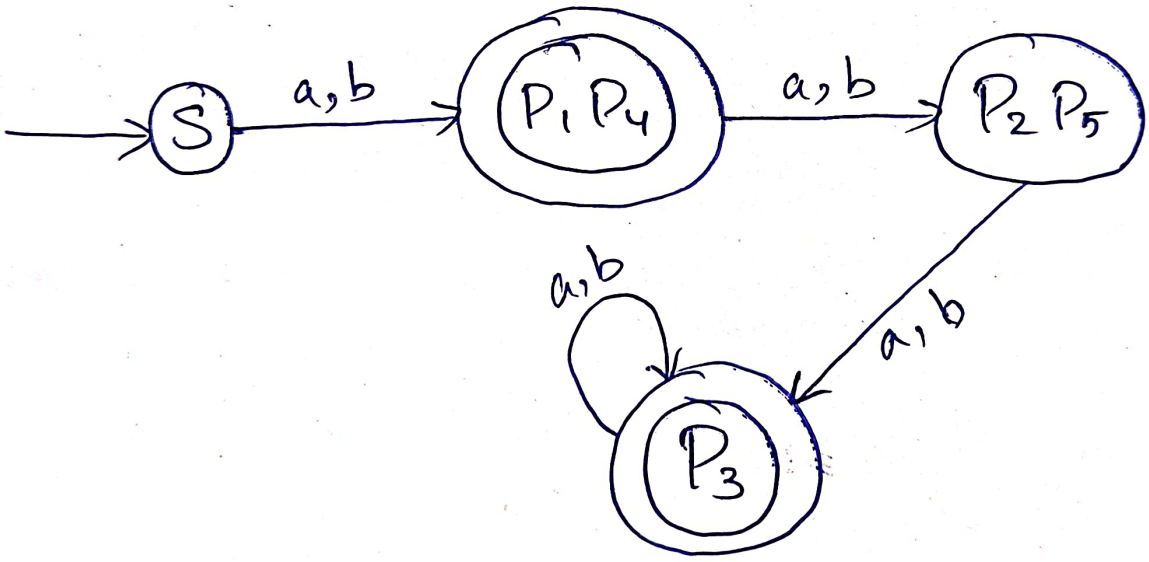
$\{S, P_2, P_5\}$ $\{P_1, P_4\}$ $\{P_3\}$

2 equivalence

$\{S\}$ $\{P_2, P_5\}$ $\{P_1, P_4\}$ $\{P_3\}$

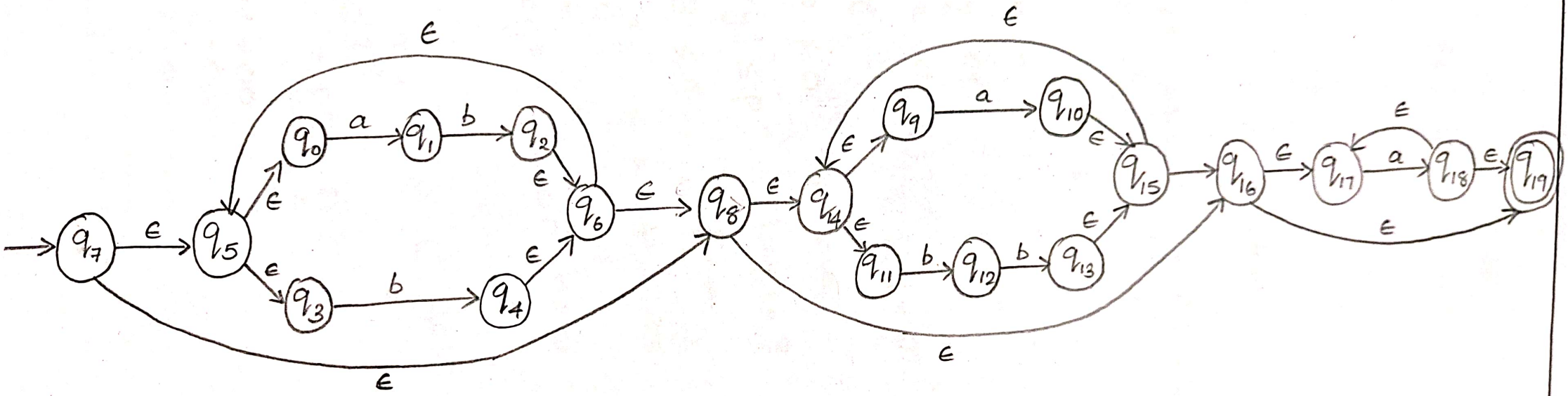
3 equivalence

$\{S\}$ $\{P_2, P_5\}$ $\{P_1, P_4\}$ $\{P_3\}$



14)
a) Develop equivalent automata for the regular expression

$(ab+b)^*(a+bb)^*a^*$



14)
b) Using Pumping Lemma for regular language prove that the language $L = \{0^n \mid n \text{ is the perfect square}\}$ is not regular.

Ans: Assume $p = 4$

$n = \{1, 4, 9, 16, \ldots\}$

Let $w = 0^4 = 0000$

Case 1:

→ $w = \underbrace{0}_{x}\underbrace{000}_{y}\underbrace{0}_{z}$

$x = 0$

$y = 00$

$z = 0$

a) $|y| > 0$ $\quad |y| = 2$

$2 > 0 \Rightarrow$ True

b) $|xy| \leq p$ $\quad |xy| = 3$

$3 \leq 4 \Rightarrow$ True

c) $xy^iz \in L \ \forall \ i \geq 0$

Let $i = 2$

$xy^2z \Rightarrow 000000 = 0^6$

$6 \notin n$

$\therefore xy^iz \notin L \ \forall \ i \geq 0$

Case 2:

→ $w = \underbrace{00}_{x}\underbrace{0}_{y}\underbrace{0}_{z}$

$x = 00$

$y = 0$

$z = 0$

a) $|y| > 0 \quad |y| = 1$

$1 > 0 \Rightarrow$ True

b) $|xy| \leq p \quad |xy| = 3$

$3 \leq 8 \Rightarrow$ True

c) $xy^iz \in L \; \forall \; i \geq 0$

Let $i = 2$

$xy^2z = 00000 = 0^5$

$5 \notin n$

$xy^iz \notin L \; \forall \; i \geq 0$

$\therefore w \notin L$

→ Case 3:

$w = \underset{x}{0}\underset{y}{0}\underset{z}{00}$

$x = 0$
$y = 0$
$z = 00$

a) $|y| > 0 \quad |y| = 1$

$1 > 0 \Rightarrow$ True

b) $|xy| \leq p \quad |xy| = 2$

$2 \leq 4 \Rightarrow$ True

c) $xy^iz \in L \; \forall \; i \geq 0$

$i = 2$

$xy^2z = 00000 = 0^5$

$5 \notin n$

$xy^iz \notin L \; \forall \; i \geq 0$

$\therefore w \notin L$

The 3 pumping conditions are not satisfied at the same time.

$\therefore$ L is not regular.

15a) State Myhill - Nerode Theorem. Prove the Language $L = \{a^n b^n$, where $n \geq 1$ is not Regular using Myhill - Nerode Theorem?

ans:- Myhill-Nerode theorem states that the following 3 statements are equivalent :-

(i) Let the set $L \subseteq \Sigma^*$ is accepted by some finite state automata

(ii) L is the union of some of the equivalence classes of right invariant Equivalence relation of finite index.

(iii) Let the equivalence relation R_L be defined by $x R_y$ iff $\forall z$ in Σ^* xz is in L, then yz is also in L. So R_L has finite index.

$L = \{a^n b^n / n \geq 0\}$ $\Sigma = \{a, b\}$

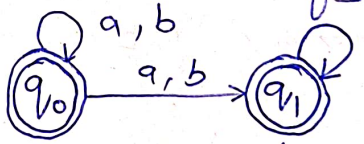
$x = a^k$ $y = a^i$ $z = b^k$ $k \neq i$

$a^n / n \geq 1$ are regular
$b^n / n \geq 1$

$a^k b^k$ will satisfy but $a^i b^k$ will not satisfy the criteria

$\therefore i \neq k$

DFA :-



$x R y \longrightarrow xz R yz$ will not be finite index class

$\therefore$ Language is not regular

b) Convert the grammar $\{S \rightarrow AaCb / ABa$, $A \rightarrow bAa/a$, $B \rightarrow BaB/b$, $C \longrightarrow C\}$ to CNF

ans:-

$S \rightarrow AaCb / ABa$
$A \rightarrow bAa/a$
$B \longrightarrow BaB/b$
$C \longrightarrow c$

① No null production
② No unit production
③ $S \to AaCb$
$S \to ABa$ $\Rightarrow$ $S \to XY$, $X \to AZ$, $Z \to a$, $Y \to CD$, $D \to b$

$S \to ABa \Rightarrow S \to AE$, $E \to BZ$, $Z \to a$

$A \to bAa \Rightarrow A \to FG$, $F \to b$, $G \to AH$, $H \to a$

$A \to a$

$B \to BaB \Rightarrow B \to IB$, $I \to BJ$, $J \to a$

$B \to b$

$C \to c$

$\therefore$ $S \to XY/AE$, $X \to AZ$, $Z \to a$, $Y \to CD$, $D \to b$, $E \to BZ$, $Z \not\to$

$A \to FG/a$, $F \to b$, $G \to AH$, $H \to a$

$B \to IB/b$, $I \to BJ$, $J \to a$

$C \to c$

16) a) Convert the context free grammer with productions: $\{S \rightarrow aSb/\epsilon\}$ into CNF/Greibach normal form.

Answer

$S \rightarrow aSb/\epsilon$

1) No left recursion.

2) Remove ϵ production

$S \rightarrow aSb/ab.$

3) No unit productions.

4) Substitution

$B \rightarrow b$

$S \rightarrow aSb/aB$

$B \rightarrow b$

16) b) Convert Context free Grammer with productions.

$S \to aSa/bSb/ss/\epsilon$ in to Chomsky Normal form.

Answer

$S \to aSa/bSb/ss/\epsilon$.

① $S' \to S$

$S \to aSa/bSb/ss/\epsilon$.

② $S \to S'$

$S \to aSa/bSb/ss/s/aa/bb$.

③ $S \to aSa/bSb/ss/aa/bb$

$S' \to aSa/bSb/ss/aa/bb$

④ $A \to a$

$B \to b$

$C \to AS$

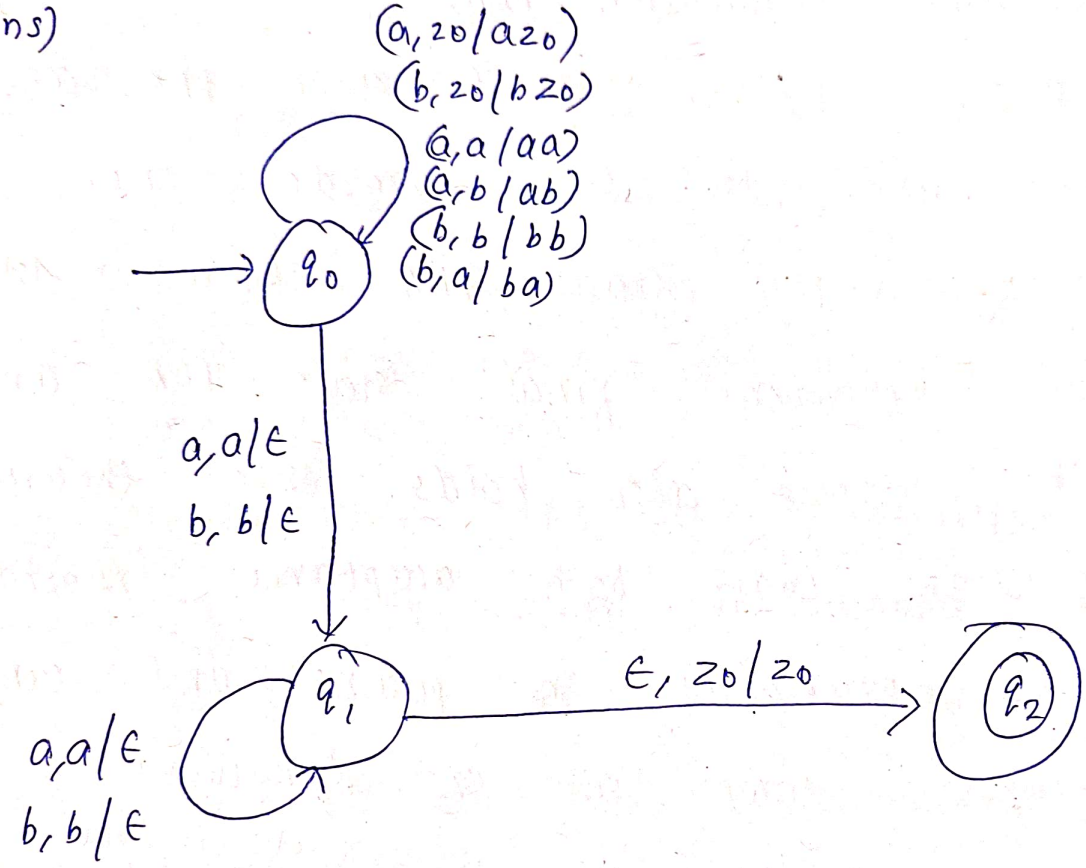
$D \to BS$

$S \to CA/DB/ss/AA/BB$

$S' \to CA/DB/ss/AA/BB$

17) a) Design a PDA for the language $L = \{ ww^r / w \in \{a,b\}^* \}$. Also illustrate the computation of the PDA on the string 'aabbaa'.

Ans)



aabbaa

$(q_0, aabbaa, z_0)$

$\downarrow$

$(q_0, abbaa, az_0)$

$\downarrow$

$(q_0, bbaa, aaz_0)$

$\downarrow$

$(q_0, baa, baaz_0)$

push

$(q_0, aa, bbaaz_0)$

$\downarrow$

$(q_0, a, abbaaz_0)$

$\downarrow$

$(q_0, \epsilon, aabbaaz_0)$ X

POP

(q_1, aa, aaz_0)

$\downarrow$

(q_1, a, az_0)

$\downarrow$

$(q_1, \epsilon, z_0) \rightarrow (q_2)$

b) State Equivalence theorem between empty stack PDA and Final state PDA.

Ans) The state Equivalence Theorem states that any language accepted by an empty stack Pushdown Automation (PDA) can also be accepted by a final state PDA, and vice versa. In other words, for every empty stack PDA, there exists an equivalent final state PDA and the reverse also holds. This theorem ensures that both acceptance mechanisms are equivalent in power and can recognize same set of languages (Context Free Languages)

Empty stack Acceptance

A PDA accepts a string if it processes the entire input and empties the stack at the end.

Final state Acceptance

A PDA accepts a string if it reaches a designated final state after processing the input, regardless of the stack's contents.

Conversion from Empty stack PDA to Final state PDA

→ To convert an empty stack PDA to a final state PDA, add a new "accept" state to PDA.

→ Modify the PDA to transition to this new state when it would normally halt with an empty stack. By reaching this final state, the PDA achieves final state acceptance.

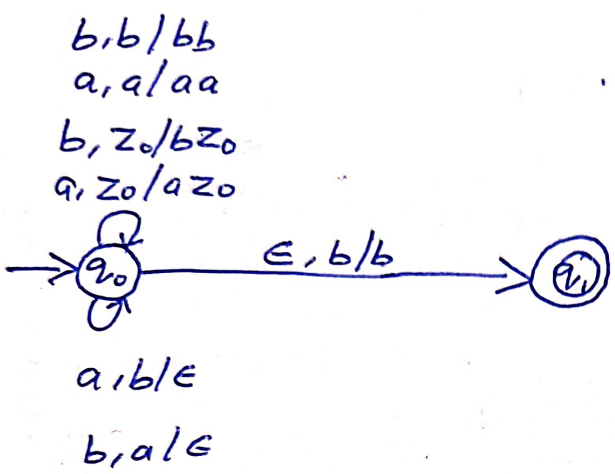
Conversion from Final state to Empty stack PDA

→ To convert a final state PDA to empty stack PDA, add a special bottom-of-stack symbol at the start of the computation.

→ Design the PDA so that it removes this special symbol only when it reaches a final state. If the input is accepted, the stack will be empty, achieving empty stack acceptance.

18(a) Design a PDA for strings in which number of a's is less than number of b's.

Ans:-



$L = \{abb, babba, abaabbb\}$

Eg:- Consider abaabbb

$(q_0, \underline{a}baabbb, Z_0)$

$(q_0, \underline{b}aabbb, \underline{a}Z_0)$

$(q_0, aabbb, Z_0)$

$(q_0, \underline{a}bbb, \underline{a}Z_0)$

$(q_0, \underline{b}bb, \underline{a}aZ_0)$

$(q_0, \underline{b}b, \underline{a}Z_0)$

$(q_0, \underline{b}, Z_0)$

$(q_0, \epsilon, \underline{b}Z_0)$

8(b) Using Pumping Lemma prove the given Language is not Context Free.

$L = \{a^n b^{2n} c^n / \text{where } n >= 1\}$

Ans:-

$$w = \underbrace{aa}_{u} \underbrace{bb}_{v} \underbrace{bb}_{x} \underbrace{c}_{y} \underbrace{c}_{z}$$

$P = 8$

$|vy| > 0 \Rightarrow 3 > 0$ True

$|vxy| \leq P \Rightarrow 5 \leq 8$ True

$K = 2$

$uv^k x y^k z = aa\ bbbbbb\ cc\ c$

$= a^2 b^6 c^3$ false

Since $uv^k xy^k z$ for $k = 2$ is false.

The Language is not Context Free Language (CFL). Hence it is Proved.

19 a) Define formally Type 0, Type 1, Type 2 and Type 3 grammar. Show the corresponding automata for each class.

Ans) Type 0 unrestricted Grammar

unrestricted grammar is defined as $G = (V, T, P, S)$

where V = finite set of non terminals

T = finite set of terminals.

S = starting symbol

P = set of production Rule.

$\alpha \rightarrow \beta$ where α, β are members of $(V \cup T)^*$ and are arbitary strings of the grammar with $\alpha \neq \varepsilon$

* unrestricted grammar do not put restriction on production Rule.
* Type 0 languages are recognized by turing Machine
* Language generated by this grammar is ~~known~~ called recursively enumerable language.

Eg:- $aBcd \rightarrow acd$.

$aSab \rightarrow ba$

$A \rightarrow S$.

Type 1 grammar (context sensitive grammar)

$G = (V, T, P, S)$

* It is a formal grammar in which the LHS and RHS of any production Rules may be surrounded by a context of terminals and non-terminals Symbols.
* CSG follows the machine linear bounded automata (LBA). The grammar G is said to be context sensitive if all the production are of the form $\alpha \rightarrow \gamma$ where

$x, y \in (V \cup T)^+$. epsilon is not allowed .

- $|x| \leq |y|$.

eg:- $AB \rightarrow aa / aaA$

$aB \rightarrow a$

$aBb \rightarrow abbc$

Type 2 Grammar (Context-free languages)

- Grammar generate context-free languages.
- The languages generated by the grammar is recognized by pushdown automata
- A grammar is said to be Context free if every Rule have a single non terminal on LHS.

Formal Definition :-

A grammar $G = \{V, T, P, S\}$ is said to be context free if the production Rule is of the form $A \rightarrow \alpha$ where A is a non terminal and α is a terminal, non terminal or ε

V: variables.

T:- Terminals

P:- Production Rule.

S :- Start Symbol.

eg:- $S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$.

Type 3 Grammar (Regular Grammar)

$G = (V, T, P, S)$

V: Variables (Non terminals)

T: Terminals.

P: Production Rule

S: Start Symbol

* Type 3 grammar restricts its rules to a single nonterminal on the left side and right hand side consist of a single terminal possibly followed (or preceded, but not both in the same grammar) by a single non terminal. A production of the form.

$A \to a$ or $A \to aB$ where A, B are $\in V$ and $a \in T$ is called Type 3. The rule $S \to \epsilon$ is also allowed here if S does not appear on the right side of any rule.

* Type 3 Languages are Recognized by finite automata

Eg:-

$x \to \epsilon$

$x \to a \mid ay$

$y \to b.$

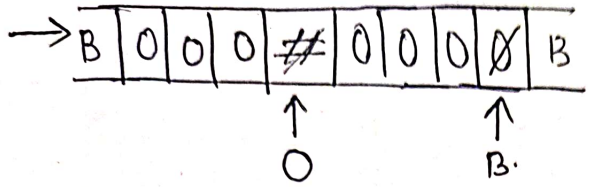
9b) Design a TM to find the sum of two numbers m and n. Assume that initially the tape contain m number of 0's followed by a # followed by n number of 0's.

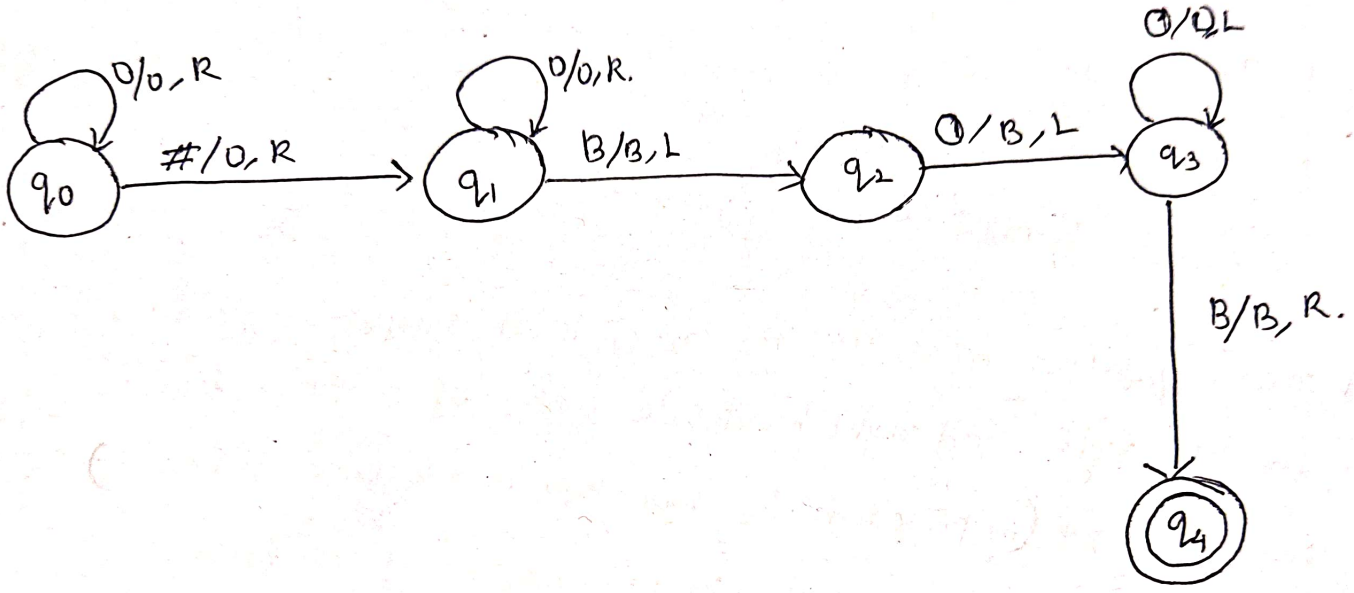
$f(m, n) = m + n$

m = 000

n = 0000

B	0	0	0	#	0	0	0	0	B


B
0
0
0
#
0
0
0
0
B
O
B.


0/0, R
#/0, R
0/0, R.
B/B, L
0/B, L
0/0, L
B/B, R.
q_0
q_1
q_2
q_3
q_4

20)

a) Design a Linear Bounded Automata for the following Language $L = \{a^n b^n c^n / n >= 1\}$

Ans $M = (Q, \Sigma, \Gamma, \delta, q_0, M_L, M_R)$

where Q = set of states

Σ = set of input signals

Γ = tape symbols

δ = transition function

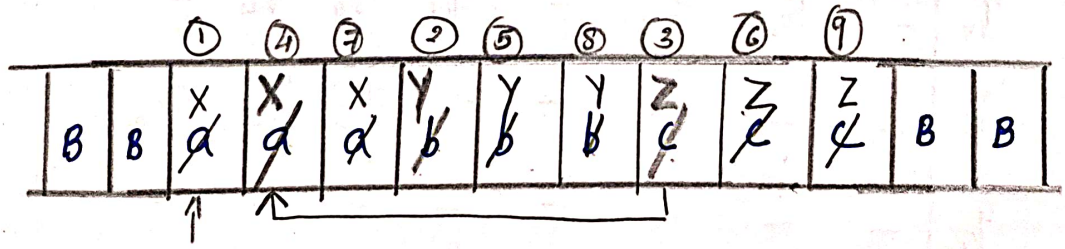
q_0 = initial state

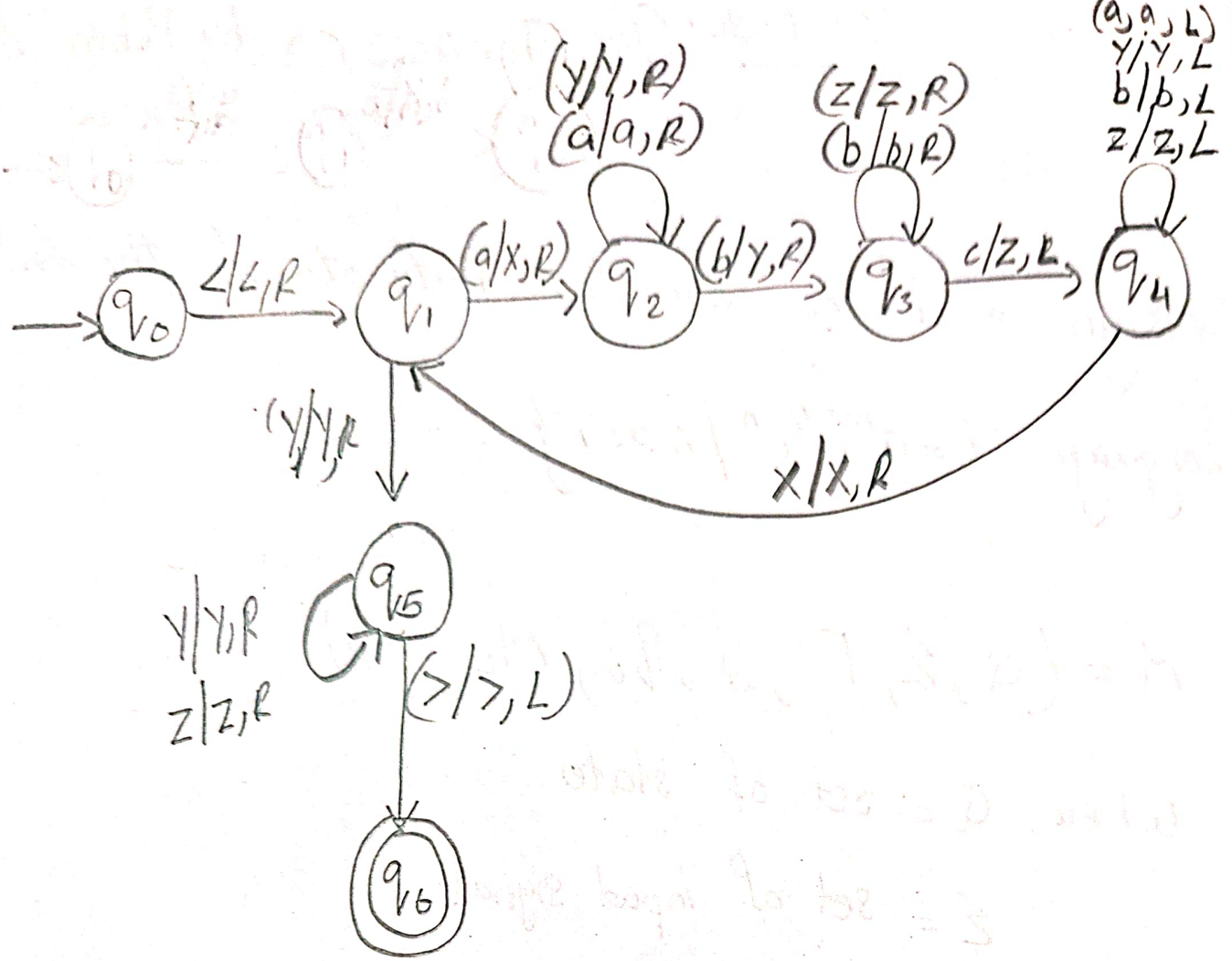
M_L = left end marker

M_R = right end marker

$\omega = \{abc, aabbcc, aaabbbccc, \ldots\}$

aaabbbccc



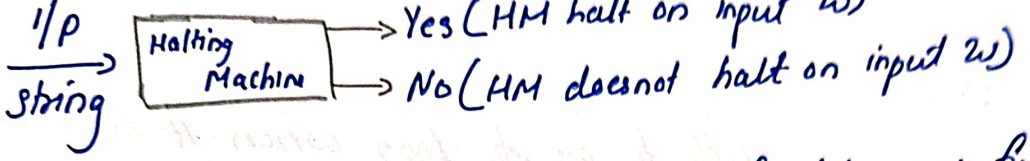


δ	<	a	b	c	X	Y	Z	>
q_0	q_1	–	–	–	–	–	–	–
q_1	–	q_2	–	–	–	q_5	–	–
q_2	–	q_2	q_3	–	–	q_2	–	–
q_3	–	–	q_3	q_4	–	–	q_3	–
q_4	–	q_4	q_4	–	q_1	q_4	q_4	–
q_5	–	–	–	–	–	q_5	q_5	q_6
q_6	–	–	–	–	–	–	–	–

20)

b) Prove that 'Turing Machine halting problem' is undecidable.

Ans: Halting Problem



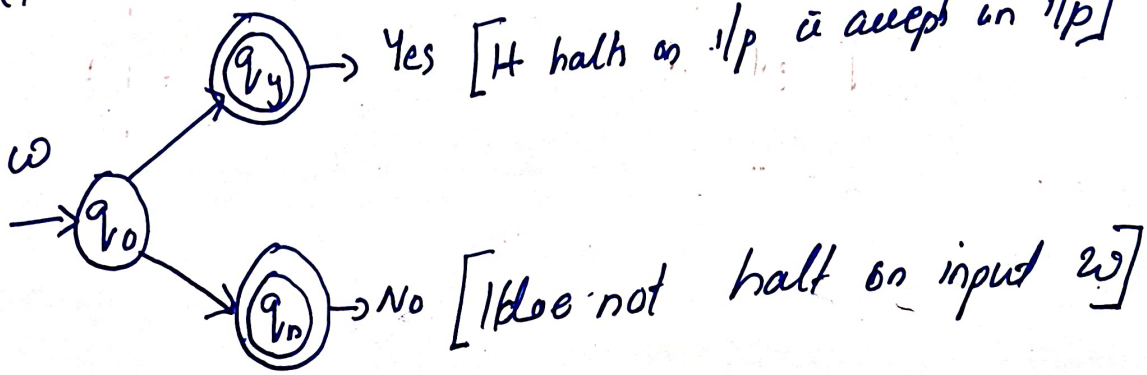
→ Halting problem is the problem of determining from a description of a machine and an input whether the program or machine will halt or continue to run forever.

Theorem

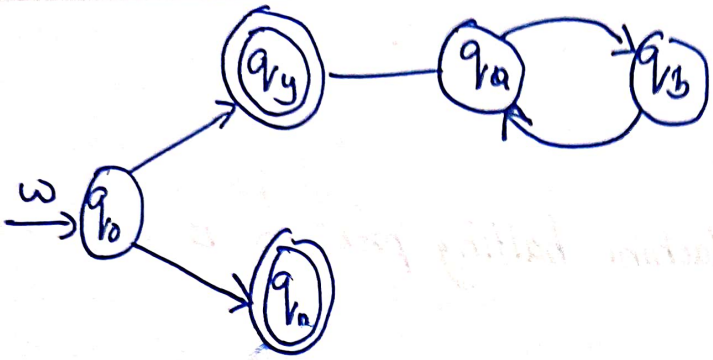
The halting problem of a TM is undecidable

Proof

1. Assume there exist an algorithm & TM 'H' that solves the problem
2. Input to 'H' is w
3. let the final state be q_y and q_n



4. Suppose adding state q_a & q_b to q_y



5. If C halts then loop forever
   else return
6. Halt loop body tells to go to loop when it does not halt, otherwise asked to return.
7) This would arise the condition that string given is undecidable
   So the halting problem is undecidable.